

★ WEEK 7 · LECTURE

Trees & Ensembles

Decision Trees, Random Forest, XGBoost — and Isolation Forest

Expanded edition · 38 slides · with three hands-on labs

The most popular tools for table data. We start with one simple tree, then combine many into a forest, and learn the boosting trick that wins competitions.

Where We Are Going Today

Weeks 4–6 gave us linear, classifiers, SVM. Trees are the workhorse for table data

1

One decision tree

A flowchart of yes-no questions. Easy to read — but prone to memorizing.

2

A forest of trees

Random Forest (vote) and XGBoost (fix mistakes): combine many trees into one strong model.

3

Which clues mattered + Isolation Forest

Measure feature importance, then meet anomaly detector #3.

By the end you will grow a tree by hand, raise a forest, and isolate the odd one out.

1

PART 1 · ONE TREE

One Tree

Decision Trees: a flowchart of questions

You Already Know This Game

A decision tree is just '20 Questions'

In '20 Questions', you narrow down an answer with yes-no questions: 'Is it alive? Bigger than a cat?' A decision tree does exactly this — asking the most useful question at each step.

The pieces

- Each question = a 'split' (e.g. income > \$50k?).
- Each path of answers leads down the tree.
- Each ending (a 'leaf') gives a decision.

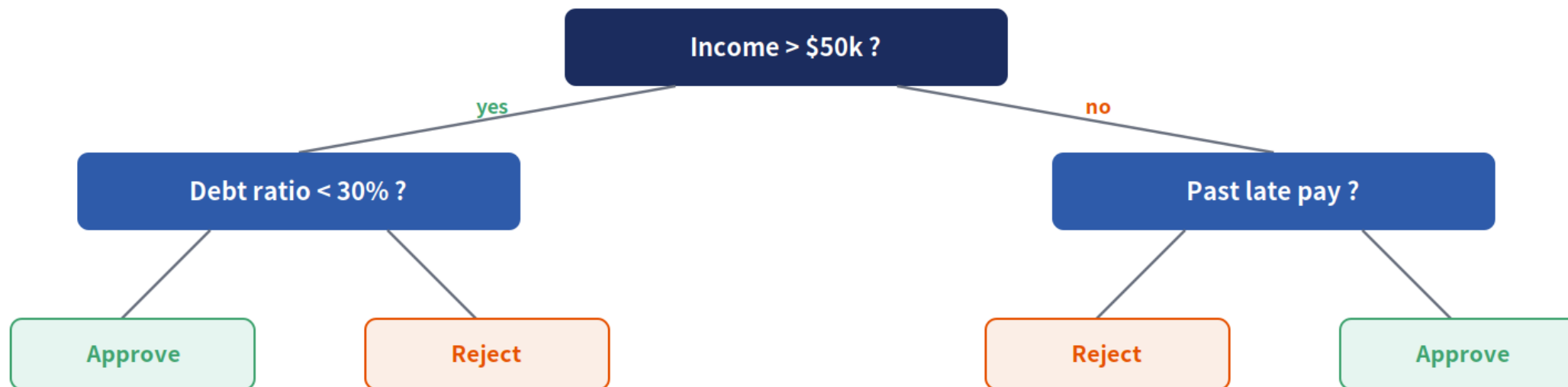
Why people love trees

- You can read the whole logic, top to bottom.
- No math needed to explain a decision.
- Great when a bank must justify 'why rejected?'

A Decision Tree in Action

Follow the yes-no answers down to a decision — fully transparent, fully traceable.

A Decision Tree Is a Game of Yes-No Questions



Follow the answers down to a leaf. Each leaf is a decision. That's the whole model —
easy for anyone to read and explain.

How a Tree Picks the Best Question

It chooses the split that makes each group the 'purest'

$$\text{Gini} = 1 - \sum_k p_k^2$$

What 'pure' means

- A group is pure if it's all one class (all 'approve' or all 'reject').
- Gini measures how MIXED a group is: 0 = pure, 0.5 = coin flip.
- p_k is the fraction of class k in the group.

The tree's strategy

- Try every possible question; keep the one with the purest children.
- Repeat on each branch, going deeper.
- Stop when groups are pure enough (or a limit is hit).

Gini by Hand — One Question's Value

Ten loans: 6 repaid, 4 defaulted. Is 'income > \$50k?' a good question? (full version: problem set)

Group	Mix (R / D)	Gini = $1 - \sum p_i^2$	Weight
Root (before asking)	6 / 4	$1 - (0.36 + 0.16) = 0.48$	—
income > 50k → YES	5 / 0	$1 - (1 + 0) = 0.00$ pure!	5/10
income > 50k → NO	1 / 4	$1 - (0.04 + 0.64) = 0.32$	5/10
After the question	—	$0.5 \cdot 0.00 + 0.5 \cdot 0.32 = 0.16$	gain 0.32

The question bought purity: $0.48 \rightarrow 0.16$. The tree tries EVERY question and keeps the biggest such gain.

One Tree's Weakness: It Memorizes

Left unchecked, a tree becomes 'Student A' again

If a tree keeps asking questions, it can make one leaf for every single training example. Then it 'remembers' the past perfectly — and fails on anything new. Classic overfitting.

How we tame one tree

- Limit its depth (how many questions).
- Require a minimum group size per leaf.
- But even tamed, a single tree is a bit shaky.

The real fix (next section)

- Don't rely on ONE tree.
- Combine MANY trees into a team.
- Their individual mistakes cancel out.

The Tree-Taming Knobs

This week's costume for the same U-curve — depth is the tree's λ

max_depth

how many questions

Depth 2–4: readable and stable.
Depth 20: one leaf per customer
— Student A with branches.

*the main dial; sweep on
validation*

min_samples_leaf

no tiny leaves

Forbid leaves with fewer than,
say, 30 examples — a rule
learned from 2 people is gossip,
not signal.

a floor against noise

the honest fix: a team

next section

Even a tamed tree is shaky: tiny
data changes reshuffle its splits.
The cure is many trees.

forest & boosting ahead

Same picture since Week 1: flexibility must be paid for with evidence.

2

PART 2 · THE FOREST

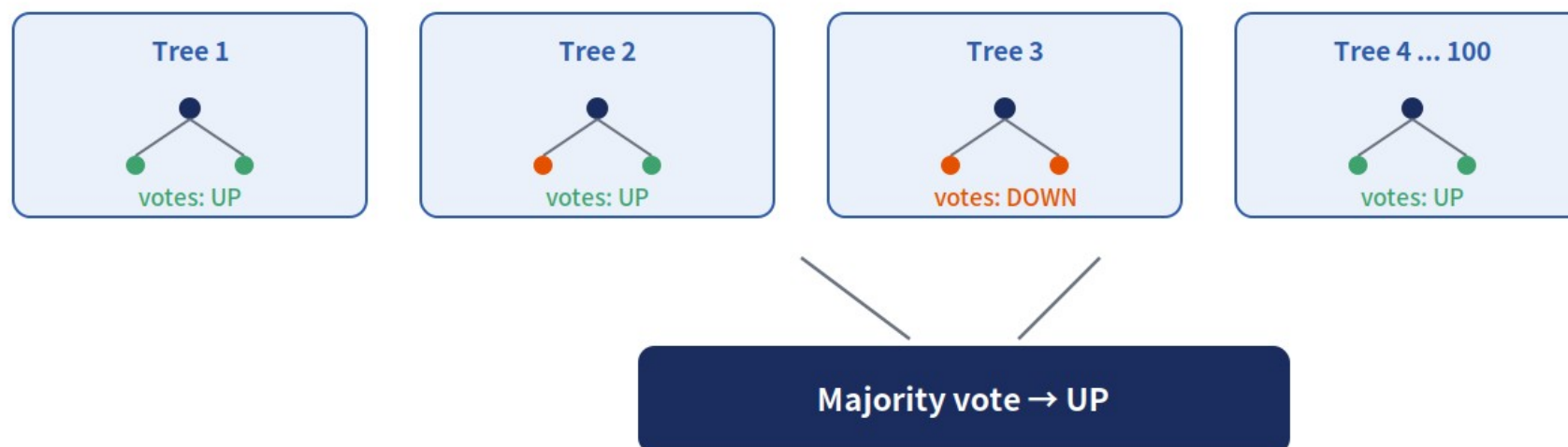
A Forest of Trees

Random Forest and XGBoost

Random Forest — the Wisdom of the Crowd

Build many slightly-different trees, then let them vote. Their errors cancel — the crowd beats any single tree.

Random Forest — Ask Many Doctors, Then Vote



Why it works

Each tree sees slightly different data & clues.

Their random mistakes cancel out in the vote.

Crowd beats one expert.

Why a Forest Beats One Tree

Two clever sprinkles of randomness

Randomness #1 — different data

- Each tree trains on a random sample of the rows.
- So no two trees see exactly the same history.
- This is called 'bagging' (bootstrap aggregating).

Randomness #2 — different clues

- At each split, a tree may only consider a random subset of features.
- This stops one strong clue from dominating every tree.
- Result: diverse trees whose mistakes don't all line up.

Diversity is the secret: independent mistakes cancel out when the trees vote together.

Bagging by Hand — Three Trees, Three Histories

Resample the same 10 loans with replacement; watch the odd row A6 appear 0, 2, then 3 times

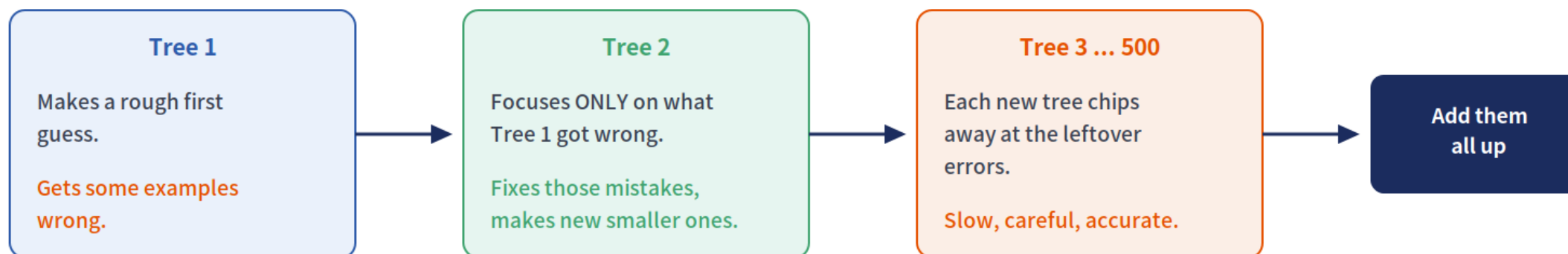
Tree	Bootstrap sample (drawn with replacement)	Times A6 appears	Consequence
T1	A1 A2 A4 A7 A9 A9	0	never meets the odd repaid-low-income case
T2	A3 A5 A6 A6 A8 A10	2	takes A6 somewhat seriously
T3	A1 A2 A6 A6 A6 A9	3	A6 dominates its low-income leaf

Same world, three remembered histories — that is where the trees' healthy disagreement comes from (full vote: problem set).

Boosting (XGBoost) — Fix the Mistakes

Build trees one at a time, each correcting the last one's errors — like a student drilling exactly what they got wrong.

Boosting (XGBoost) — Each Student Fixes the Last One's Mistakes



Like a student who, after each practice test, studies exactly the questions they got wrong.

Random Forest builds trees in PARALLEL and votes. Boosting builds them in SEQUENCE, each fixing the last.

XGBoost is the famous, fast version of boosting.

Boosting, as a Tiny Formula

The new model = old model + a small correction

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

Reading it

- F is the whole model so far; the subscript m is the step number.
- h_m is a new little tree trained on the leftover errors.
- η (the learning rate) is how big a step we take — small and careful.

Why small steps?

- Tiny corrections avoid over-reacting to noise.
- Hundreds of small steps add up to high accuracy.
- Too many steps → overfitting, so we watch validation (η + n_trees are the knobs).

Boosting by Hand — Watch the Error Melt

Four loans, target y = default indicator, $\eta = 0.5$ (full version: problem set)

Step	Predictions F for $y = (0, 0, 1, 1)$	SSE	What happened
$F_0 = \text{mean}$	(0.50, 0.50, 0.50, 0.50)	1.00	know nothing — predict the average
$F_1 = F_0 + 0.5 \cdot h_1$	(0.25, 0.25, 0.75, 0.75)	0.25	h_1 learned the residuals; take half the step
$F_2 = F_1 + 0.5 \cdot h_2$	(0.125, 0.125, 0.875, 0.875)	0.0625	the leftover error, corrected again

Each careful half-step keeps only 25% of the error — geometric convergence from two lines of arithmetic.

Bagging vs. Boosting

Committee or relay team: the forest reduces wobble (variance); boosting reduces error (bias). Both beat a lone tree.

Bagging vs. Boosting — Committee vs. Relay Team

Bagging — Random Forest

A committee voting at once.

Trees built in PARALLEL, independently.

Each sees random data + random clues.

Goal: reduce VARIANCE (less wobble).

Hard to overfit, easy to run in parallel.

Safe, sturdy, a great default.

Boosting — XGBoost

A relay team fixing mistakes.

Trees built in SEQUENCE, one by one.

Each new tree fixes the last one's errors.

Goal: reduce BIAS (more accurate).

Usually the most accurate on tables...

...but can overfit if pushed too hard.

The Algorithm That Wins Competitions

Why XGBoost quietly powers so much of finance

"For years, a huge share of winning solutions on Kaggle (the ML competition site) used gradient boosting on tabular data."

Why it dominates tables

- Most finance data is tabular (rows & columns).
- Boosting captures complex, non-linear patterns.
- Handles mixed feature types gracefully; fast and battle-tested.

The honest caveat

- On the noisy, ever-changing market (Week 1), even XGBoost can overfit.
- It is powerful, not magic.
- Careful validation (Week 10) still rules.

Committee vs. Relay Team, at a Glance

Same trees, opposite philosophies — keep this table next to the picture

	Random Forest (bagging)	Boosting (XGBoost)
How trees are built	In parallel, independently	In sequence, each fixing the last
What it mainly reduces	Variance (the wobble)	Bias (the systematic miss)
Key knobs	n_estimators, max features	eta, n_estimators, depth
Temperament	Hard to overfit, forgiving	Sharper — and needs watching
Reach for it when	You want a robust baseline fast	You need every drop of accuracy

In practice: start with a Forest for a sanity baseline, then let Boosting chase the residual accuracy.

3

PART 3 · IMPORTANCE

Which Clues Mattered?

Feature importance: MDI, MDA, SFI

Three Ways to Rank the Clues

Trees can tell us which features drove the decisions. MDA (the shuffle test) is the most trustworthy; MDI is free but biased.

Which Clues Mattered? Three Ways to Measure

MDI · built-in

"How much did each clue help the splits?"

Comes free from the trees.

Adds up how often a clue was used to split, and how well.

+ Instant, no extra work.

- Biased toward clues with many possible values.

MDA · shuffle test

"Shuffle one clue. Does accuracy drop?"

Scramble a clue's column, re-check the score.

Big drop = that clue mattered.

+ Honest, model-agnostic.

- Slower; shaky when clues are correlated.

SFI · one at a time

"How good is each clue all by itself?"

Train a tiny model on just one clue; score it.

Repeat for every clue.

+ Not fooled by teamwork.

- Misses clues that only help in combination.

Why Importance Matters in Finance

It's not just nice-to-have — it's a safety check

A model that scores well for a silly reason is a time bomb. Feature importance tells you WHICH clues drove it — so you can sanity-check the story.

Good uses

- Spot leakage: is a 'too good' clue secretly the answer?
- Trim the factor zoo (Week 4) down to what matters.
- Explain the model to risk officers and regulators.

A warning

- Importance is not proof of cause (Week 14).
- Correlated clues can steal each other's credit.
- Always read importance with judgment, not blind faith.

MDI vs. MDA vs. SFI

Three rulers for the same question — each with a price and a bias

Method	How it measures a clue	Cost	Watch out for
MDI (in-tree)	Total purity it bought across all splits	Free — computed during training	Biased toward many-valued, correlated clues
MDA (shuffle test)	How much scores DROP when the clue is scrambled	Extra passes over validation data	Splits credit among correlated cousins
SFI (solo test)	Model's score using that single clue alone	One model per clue — expensive	Misses clues that only work in combination

Course habit: quote MDA, sanity-check with MDI, reach for SFI when the stakes justify the bill.

4

PART 4 · ISOLATION FOREST

The Easy-to-Isolate Outlier

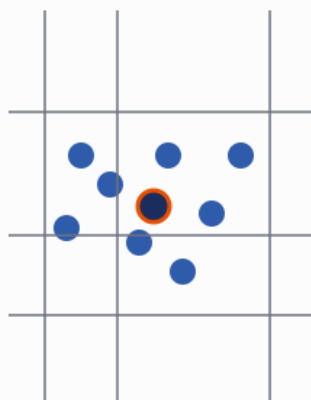
Isolation Forest — anomaly detector #3

Isolation Forest — Cut Until Alone

Flip trees around: randomly cut until each point is isolated. Anomalies sit in empty space — few cuts give them away.

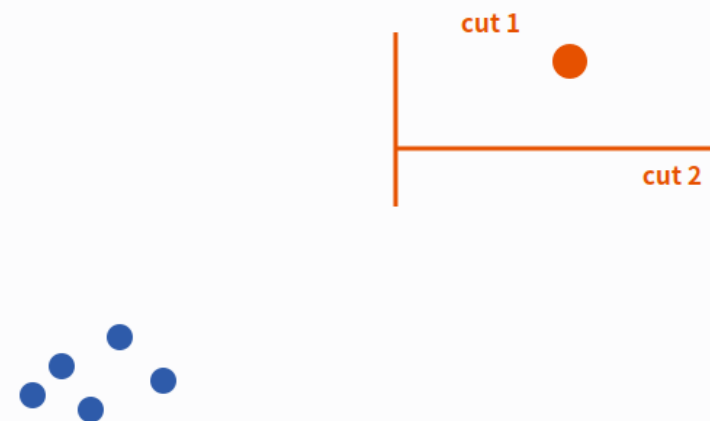
Isolation Forest — Odd Points Are Easy to Cut Off

A normal point: hard to isolate



needs MANY random cuts to fence off

An anomaly: isolated in a few cuts



a couple of cuts already fence it off alone

Fewer cuts needed to isolate a point → more likely an anomaly. No labels required.

One Cut Is Usually Enough

Five account balances — four clustered, one far out. Cut at a random point of the range

Balances (\$k)	10, 11, 12, 13 and 45	Range = [10, 45]
Chance the FIRST random cut isolates 45	gap (13 → 45) over range = 32 / 35	≈ 91%
Chance it isolates 11 (an interior point)	needs cuts landing in (10,11) AND (11,12)	tiny — takes many cuts

The whole algorithm in one sentence

- Average the number of random cuts needed to isolate each point over many random trees — a SHORT path means odd. No labels, no distances, just cuts: that is why it flies on big data.

Our Four Anomaly Detectors, Side by Side

Isolation Forest joins the family — each finds 'odd' differently

Detector	Core idea	Best when	Week
LOF	Low local density (a loner)	Mixed dense & sparse normal	5
OCSVM	Fence around normal (margin)	High-dimensional, stable	6
Isolation Forest	Few cuts to isolate	Large data, very fast	7
Autoencoder	Hard to reconstruct	Complex, non-linear patterns	8

No single detector is best. Banks run several together so each covers the others' blind spots.

How a Real Fraud System Stacks These

A production pipeline, stage by stage

Stage 1 — fast screen

iForest + OCSVM

Scan every transaction; cheap and fast. Flag anything unusual, pass the clearly-normal instantly.

anomaly detectors up front

Stage 2 — sharp model

XGBoost

Scores the flagged ones for fraud risk — an accurate probability. (A time-aware LSTM joins in Week 8.)

the accuracy engine

Stage 3 — explain

SHAP (Week 14)

'Flagged because amount + location + hour.' Humans review with a clear reason, not a black box.

the trust layer

Screen fast → score sharply → explain clearly. Every tool from this course has a place.

Which Tree Tool? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Which tool?
A regulator asks to trace one rejection, step by step	A single shallow tree — readable top to bottom
Your one tree changes wildly with each data refresh	Random Forest — bagging averages the wobble away
Baseline is fine; you need the last drop of accuracy	Boosting — small sequential corrections (mind η)
A 'too good' feature smells like leakage	MDA shuffle test — scramble it and watch the score
Ten million rows, no labels, need anomalies by lunch	Isolation Forest — random cuts, linear speed

If you got 4 of 5, the toolbox of Week 7 is yours.

5

PART 5 · HANDS ON

Your Turn — Hands On

Grow trees, raise a forest, isolate the odd one out

Getting This Week's Data

Companion notebook — reuse the § 5 classification set

From the companion notebook (§ 5)

- Run Setup (§ 0), then the § 5 offline cell.
- Gives X (clues) and yb (label, ~1% positive).
- Same data as Weeks 5–6 — easy to compare, no login.

What we'll do

- Lab 1: Tree vs Forest vs Boosting + importance.
- Lab 2: Isolation Forest finds anomalies, no labels.
- XGBoost-style boosting via sklearn (always available).

[CODE] FML_Data_Setup_Guide.ipynb · § 5 offline classification set

```
from sklearn.datasets import make_classification
X, yb = make_classification(n_samples=20000, n_features=20, n_informative=6,
                           weights=[0.99, 0.01], random_state=42)
# X = clues, yb = label (about 1% positive, fraud-like)
```

Hands-On Lab 1 — Tree vs. Forest vs. Boosting

Compare all three, then see which clues mattered

[CODE] colab_lab1.py · data from setup guide § 5

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier # ~XGBoost
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
import numpy as np

Xtr, Xte, ytr, yte = train_test_split(X, yb, random_state=0)
for name, m in [('Tree', DecisionTreeClassifier(max_depth=4)),
                ('Forest', RandomForestClassifier(n_estimators=200)),
                ('Boost', GradientBoostingClassifier())]:
    m.fit(Xtr, ytr)
    print(name, round(f1_score(yte, m.predict(Xte)), 3))

rf = RandomForestClassifier(200).fit(Xtr, ytr)
print('top clues:', np.argsort(rf.feature_importances_)[-5:])
```

Your task (step by step)

- Load X, yb (setup guide § 5).
- Split into train and test.
- Fit a single Tree, a Forest, and Boosting.
- Compare their F1 scores.
- Print the Forest's top importances.
- Q: does the team beat the lone tree?

Hands-On Lab 2 — Isolation Forest Anomalies

Find the odd ones with random cuts — no labels

[CODE] colab_lab2.py · data from setup guide § 5

```
import numpy as np
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.01, random_state=0)
flag = iso.fit_predict(X)      # -1 = anomaly, 1 = normal
is_anom = (flag == -1)

# trained WITHOUT labels; now check against yb
caught = np.sum(is_anom & (yb == 1))
total = np.sum(yb == 1)
print(f'Isolation Forest flagged {is_anom.sum()} points')
print(f'caught {caught} of {total} real anomalies,',
      'no labels used')
```

Your task (step by step)

- Use X only (ignore labels yb).
- Fit Isolation Forest to flag ~1%.
- NOW peek at yb to check.
- How many real anomalies did it catch?
- Q: how does it compare to LOF (W5) and OCSVM (W6)?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 loop above, one plain-English sentence per line.
- 2) Sweep DecisionTree max_depth from 1 to 20; plot train F1 and validation F1 on one chart and mark where the two curves part ways (the U-curve).
- 3) Compute MDA importance: shuffle each feature on the validation set and record the F1 drop; compare the ranking with rf.feature_importances_ (MDI).
- 4) Combine three detectors - LOF, OCSVM, IsolationForest - by majority vote on 8000 rows; report catches and flags vs each detector alone.
- 5) In 3 sentences: why does the vote beat any single detector, in this week's diversity language?

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 How does Random Forest differ from Boosting?

- A Forest builds trees in sequence to fix errors
- B Forest votes in parallel; boosting fixes errors in sequence ✓**
- C They are the same
- D Forest uses no trees

Answer: Bagging (Forest) = parallel + vote; Boosting (XGBoost) = sequential, each tree fixes the last's mistakes.

Q2 Why does Isolation Forest flag a point as anomalous?

- A It has the highest value
- B Very few random cuts isolate it ✓**
- C It appears most often
- D It has a label

Answer: Anomalies sit in empty space, so few random cuts isolate them — a short isolation path means odd.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 In boosting, a SMALL learning rate η means:

- A** Bigger, faster corrections
- B** Tiny careful steps — more trees, less noise-chasing ✓
- C** The trees grow deeper
- D** Bagging is enabled

Answer: Small η takes cautious steps toward the residuals; hundreds of them add up without over-reacting to noise.

Q4 A feature ranks #1 on MDI but its MDA (shuffle) drop is ~0. Most likely:

- A** It is the true cause
- B** MDI's bias — the clue bought splits, not skill ✓
- C** The model is perfect
- D** Shuffling is broken

Answer: MDI rewards many-valued, correlated clues during training; if scrambling it costs nothing, it carried no real signal.

Three Ideas, One Picture Each

The theme: one flexible learner is fragile — a diverse, disciplined team is not

1

A tree = 20 Questions with a purity meter

Gini picks each split ($0.48 \rightarrow 0.16$ in our mini-walk). Readable — but alone it memorizes.

2

Forest = committee, Boosting = relay

Bagging votes wobble away; boosting melts error geometrically (SSE $1.00 \rightarrow 0.25 \rightarrow 0.0625$).

3

Importance + Isolation Forest

MDA is the trustworthy ranking; iForest flags whoever one random cut can strand alone.

Keep the pictures: 20 Questions, a voting committee, a relay team — and a lone point, one cut away.

Week 7 — Three Things to Remember

Before next class: finish Labs 1–3 and compare catches across LOF, OCSVM, Isolation Forest

1

A tree is a flowchart of questions

Readable and intuitive (Gini picks each split), but one tree alone tends to memorize and overfit.

2

Forests and boosting combine trees

Random Forest votes (parallel, less wobble); XGBoost fixes mistakes (sequential, more accurate).

3

Importance + Isolation Forest

MDI/MDA/SFI rank the clues; Isolation Forest flags anomalies by how few cuts isolate them.

Next week — Neural Networks & Deep Learning: the perceptron, backprop (+ Autoencoder, #4).